qnode/.style
qnode/.style//
qcore/.style
qcore/.style/.@cmd
qcore/.style/.@cmd
qcore/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=qnode, draw=qblue, line width=1.1pt, fill=qlightblue, minimum width=35mm
qinput/.style
qinput/.style/.@cmd
qinput/.style/.@cmd
qinput/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=qnode, draw=qgreen, fill=qlightgreen
qaction/.style
qaction/.style/.@cmd
qaction/.style/.@cmd
qaction/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=qnode, draw=qorange, fill=qlightorange
qdanger/.style
qdanger/.style/.@cmd
qdanger/.style/.@cmd
qdanger/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=qnode, draw=qred, fill=qlightred
qarrow/.style
qarrow/.style/.@cmd
qarrow/.style/.@cmd
qarrow/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=-Latex[length=2.2mm], thick, draw=qdark
qbluearrow/.style
qbluearrow/.style/.@cmd
qbluearrow/.style/.@cmd
qbluearrow/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font=-Latex[length=2.2mm], thick, draw=qblue
qgreenarrow/.style
qgreenarrow/.style/.@cmd
qgreenarrow/.style/.@cmd
qgreenarrow/.style/.@cmddraw=qdark, rounded corners=2pt, thick, align=center, minimum height=12mm, minimum width=30mm, fill=white, font= -Latex[length=2.2mm], thick, draw=qgreen

QRAEMON

Software Product

From Zero to a Successful Real-World Product

Architecture, Engineering, Validation, Deployment,
Continual Learning, Memory, Pilots and Productization

Product:	QRAEMON Software
Product class:	Robot-agnostic cognitive runtime
Primary objective:	Persistent experience-driven intelligence for existing robotic systems
Deployment:	Robot computer / workstation / edge computer / simulation
Architecture:	Shared cognitive core with robot-specific adapters

Technical product-development specification
Version 1.0

Contents

1 Executive Summary **4**

2 Product Definition **4**
2.1 What QRAEMON Software Is . . . 4
2.2 What QRAEMON Software Is Not . . . 5

3 Product Thesis **5**

4 Product Architecture **5**

5 Core Cognitive Loop **6**

6 Development Philosophy **6**
6.1 Interface Before Intelligence . . . 6
6.2 Simulation Before Physical Deployment . . . 7
6.3 Measurement Before Claims . . . 7
6.4 Memory Before Scale . . . 7
6.5 Safety Outside the Model . . . 7
6.6 Robot Agnosticism . . . 7

7 Phase 0 — System Specification **7**
7.1 Canonical Robot State . . . 7
7.2 Canonical Action Schema . . . 8
7.3 Action Lifecycle . . . 8

8 Robot Adapter SDK **8**

9 Software Repository Architecture **9**

10 Runtime Services **10**

11 Memory Architecture **10**
11.1 Experience Representation . . . 10
11.2 Experience Lifecycle . . . 11

12 Memory Implementation Order **11**
12.1 Version 1 — Raw Episodic Memory . . . 11
12.2 Version 2 — Retrieval . . . 11
12.3 Version 3 — Selective Memory . . . 11
12.4 Version 4 — Consolidation . . . 11
12.5 Version 5 — Forgetting . . . 12

13 Reasoning and Planning **12**

14 Safety Architecture **12**
14.1 Safety Responsibilities . . . 12
14.2 Safety Relationship . . . 13

15 Failure Handling **13**

16 Observability and Decision Tracing **13**

17 Phase 1 — Simulation **14**
17.1 Initial Tasks . . . 14

18 No-Memory Baseline **15**

19 Phase 2 — Memory Proof **15**

20 Memory Ablation Program **16**

21 Phase 3 — Physical MVP **16**

22 Hardware-in-the-Loop Validation **16**

23 Production Software Architecture **17**
23.1 Core Components . . . 17
23.2 Versioning . . . 17

24 Testing Strategy **18**
24.1 Unit Tests . . . 18
24.2 Integration Tests . . . 18
24.3 Regression Tests . . . 18

25 Security Architecture **18**

26 Deployment Engineering **19**

27 Developer SDK **19**

28 Evaluation Platform **20**

29 Continual Learning **20**

30 Cross-Task Transfer **20**

31 Cross-Robot Experience **21**

32 External Pilot Program **21**

33 Productization **21**

34 Software Product Packaging **22**

35 Final Software Roadmap **22**
35.1 Gate 1 — Architecture . . . 22
35.2 Gate 2 — Simulation . . . 22
35.3 Gate 3 — Physical MVP . . . 23
35.4 Gate 4 — Experience Proof . . . 23
35.5 Gate 5 — Pilot . . . 23
35.6 Gate 6 — Productization . . . 23

36 Definition of Software Success **24**

37 Core Software Metrics **24**

38 Immediate Engineering Backlog **25**

39 What Not to Build First 25

40 Transition to QRAEMON Embedded 26

41 Long-Term Software Moat 26

42 Final Architectural Position 27

1 Executive Summary

QRAEMONSoftware is the first deployable form of the QRAEMON artificial-brain architecture.

It is not intended to replace a robot's servo controller, emergency-stop system, low-level actuator controller, or hard safety layer.

Instead, it operates above those systems.

Its responsibility is to:

- perceive relevant robot and environmental state;
- construct the current cognitive context;
- retrieve relevant accumulated experience;
- reason about the current situation;
- select strategies and skills;
- issue bounded task-level or skill-level actions;
- observe the result;
- record the experience;
- evaluate whether the experience was useful;
- improve future decisions through accumulated experience.

The existing robot control stack remains responsible for deterministic physical execution and safety.

> **Core Principle**
>
> **The first product is not "an AI model controlling a robot."**
> It is a persistent cognitive runtime that sits above an existing robot control system and gradually becomes better at making task-level decisions because it remembers and learns from operational experience.

The architecture follows the established QRAEMON principle that the same cognitive core should eventually support many robot embodiments. The robot-specific translation is isolated inside an adapter layer. :chatgpt-content-referenceindex="1"

2 Product Definition

2.1 What QRAEMON Software Is

QRAEMON Software is a deployable software runtime capable of running on:

- an existing robot industrial computer;
- a permitted robot controller computer;
- a workstation;
- an edge computer;
- a research server;
- a simulation environment.

The product consumes canonical robot state and perception information through a robot-specific adapter and returns bounded task or skill commands.

This is the software-first path because it avoids designing custom hardware before the cognitive architecture has been experimentally validated.

2.2 What QRAEMON Software Is Not

The first version should explicitly **not** be:

- a servo controller;
- an emergency-stop system;
- a motor-current generator;
- a replacement for hard safety controls;
- a cloud-dependent robot brain;
- a giant sensor-data database;
- a generic chatbot;
- a generic RAG product;
- a requirement to retrain a giant model after every experience.

> **Engineering Warning**
>
> QRAEMON must never obtain authority over safety-critical physical execution merely because its cognitive model is more capable. Safety authority remains independent from cognitive inference.

3 Product Thesis

The core hypothesis is:

> *A robot should become measurably better at future decisions because it has experienced and learned from previous situations.*

The central engineering question is therefore not:

> "Can we make a larger robotics model?"

It is:

> "Can persistent, selective, outcome-linked experience improve future robot decisions under controlled experimental conditions?"

This distinction determines the entire development strategy.

4 Product Architecture

The software architecture is divided into six major layers:

1. robot interface;
2. perception and context;

3. memory and experience;
4. reasoning and planning;
5. action and safety;
6. learning and observability.

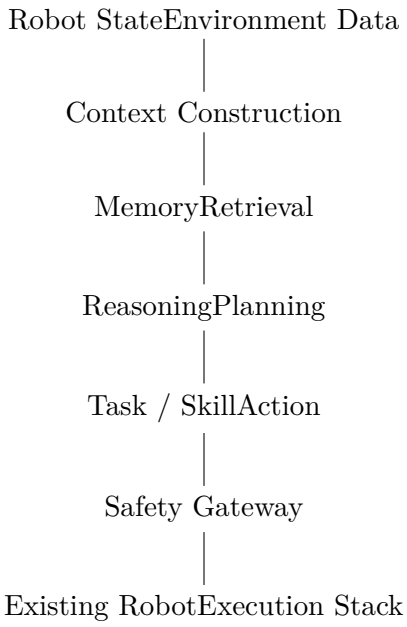


Figure 1: QRAEMON Software cognitive execution path.

Every connection is a single vertical path. No feedback line crosses another box or execution line.

5 Core Cognitive Loop

The runtime should implement a clear repeated cycle:

Observe — Context — Remember — Reason — Act — Evaluate

Figure 2: Forward cognitive loop.

The experience generated by evaluation becomes input to future memory formation.
The logical loop is therefore:

$$\text{Observe} \rightarrow \text{Context} \rightarrow \text{Remember} \rightarrow \text{Reason} \rightarrow \text{Act} \rightarrow \text{Evaluate} \rightarrow \text{Learn}$$

6 Development Philosophy

QRAEMON Software should be developed according to several non-negotiable engineering principles.

6.1 Interface Before Intelligence

The canonical interfaces should be frozen before the system becomes model-heavy.

6.2 Simulation Before Physical Deployment

The cognitive loop should first work inside a deterministic simulation.

6.3 Measurement Before Claims

The system should not be called “learning” merely because it stores information.

A learning claim requires measurable improvement.

6.4 Memory Before Scale

The first research advantage should come from experience architecture, retrieval, consolidation and continual learning rather than model size.

6.5 Safety Outside the Model

The safety layer should remain independent of cognitive inference.

6.6 Robot Agnosticism

The cognitive core should not contain robot-specific assumptions.

Robot-specific knowledge belongs in adapters and embodiment interfaces.

7 Phase 0 — System Specification

Objective: define the system before building intelligence.

7.1 Canonical Robot State

Define a common state representation containing, where available:

- joint positions;
- joint velocities;
- joint efforts;
- end-effector pose;
- end-effector velocity;
- robot mode;
- actuator state;
- sensor timestamps;
- perception observations;
- task state;
- environment state;
- fault state;
- capability state.

All units and coordinate frames must be explicit.

7.2 Canonical Action Schema

Actions should initially operate at task or skill level.
Examples:

- move-to-pose;
- grasp;
- release;
- pick;
- place;
- inspect;
- insert;
- retry;
- recover;
- execute skill.

7.3 Action Lifecycle

Every action must have an explicit lifecycle:

PROPOSED —— AUTHORIZED —— EXECUTING —— SUCCEEDED

Figure 3: Primary action lifecycle.

Failure and cancellation are explicit terminal states:

- FAILED;
- CANCELLED.

8 Robot Adapter SDK

The adapter is the embodiment translation layer.
It is not the cognitive brain.
The adapter translates:

- robot-specific state into canonical QRAEMON state;
- canonical QRAEMON actions into robot-specific commands;
- robot events into QRAEMON events;
- robot capabilities into canonical capability descriptions.

A minimal adapter contract should support:

1. `connect();`
2. `disconnect();`
3. `get_state();`
4. `get_capabilities();`

5. `execute_skill()`;
6. `cancel_action()`;
7. `get_action_status()`;
8. `get_events()`;
9. `health_check()`.

Optional interfaces may expose perception or state-estimation hooks.

> **Core Principle**
>
> A new robot should ideally require a new adapter, not a new QRAEMON cognitive architecture.

9 Software Repository Architecture

A recommended initial repository is:

```
qraemon/
|
+-- core/
|   +-- context/
|   +-- perception/
|   +-- memory/
|   +-- reasoning/
|   +-- planning/
|   +-- decision/
|   +-- learning/
|
+-- robot/
|   +-- canonical_state/
|   +-- action_api/
|   +-- capability_api/
|   +-- adapter_sdk/
|   +-- adapters/
|
+-- safety/
|   +-- policy/
|   +-- validation/
|   +-- authorization/
|
+-- runtime/
|   +-- scheduler/
|   +-- event_bus/
|   +-- telemetry/
|   +-- health/
|
+-- storage/
|   +-- experience/
|   +-- configuration/
|   +-- model_registry/
|
+-- simulation/
|
+-- evaluation/
|
```

```
+-- tools/
|
+-- tests/
|
+-- deployment/
|   +-- software/
|   +-- embedded/
|
+-- docs/
```

The exact process topology may later be simplified for constrained deployment, but the logical interfaces should remain intact.

10 Runtime Services

The first production-oriented implementation should separate responsibilities.

Recommended services:

- `qraemon-runtime`: cognitive orchestration;
- `state-service`: canonical state;
- `memory-service`: persistent experience;
- `adapter-service`: robot communication;
- `safety-gateway`: validation and authorization;
- `telemetry-service`: logs and observability.

The system should support process isolation so that failure in a cognitive component does not automatically imply loss of robot safety.

11 Memory Architecture

Memory is the central differentiating research component of QRAEMON.

The first implementation should not attempt to reproduce a complete biological brain.

Instead, it should establish an experimentally measurable experience pipeline.

11.1 Experience Representation

Each experience should contain enough information to answer:

- What situation occurred?
- What task was being attempted?
- What state was observed?
- What action was selected?
- Why was the action selected?
- What happened?
- Was it successful?
- What failure occurred?
- What recovery was effective?
- How transferable is the experience?

11.2 Experience Lifecycle

Experience —— Encode —— Retrieve —— Rank —— Consolidate —— Forget

Figure 4: Initial QRAEMON experience lifecycle.

The important point is that memory is an active computational process rather than passive storage.

12 Memory Implementation Order

Do not attempt to build the final memory architecture immediately.
Build it experimentally.

12.1 Version 1 — Raw Episodic Memory

Store complete experiences.
Goal:

- prove recording;
- prove persistence;
- prove replay;
- establish schemas.

12.2 Version 2 — Retrieval

Retrieve experiences relevant to the current context.
Goal:

- contextual similarity;
- task similarity;
- failure similarity;
- outcome relevance.

12.3 Version 3 — Selective Memory

Rank memories using factors such as:

- relevance;
- success;
- recurrence;
- novelty;
- transfer value;
- recency;
- reliability.

12.4 Version 4 — Consolidation

Merge repeated experiences into more general knowledge.

12.5 Version 5 — Forgetting

Remove, compress, archive or reduce access to memories that no longer justify their computational cost.

> **Engineering Warning**
>
> Do not equate "larger memory" with "better intelligence." QRAEMON should eventually demonstrate that selective memory produces better decisions than indiscriminate accumulation.

13 Reasoning and Planning

The initial planner should remain deliberately narrow.

It should:

1. inspect current state;
2. inspect task objective;
3. retrieve relevant experience;
4. generate candidate strategies;
5. evaluate candidates;
6. select a bounded action sequence;
7. pass the result through the safety gateway;
8. execute through the adapter.

The architecture should permit future replacement of reasoning modules without changing the robot interface.

14 Safety Architecture

Safety must be structurally independent from cognitive inference.

14.1 Safety Responsibilities

The safety layer should validate:

- allowed action;
- workspace;
- velocity;
- acceleration;
- force and torque;
- robot state;
- actuator state;
- action authorization;
- emergency conditions.

14.2 Safety Relationship

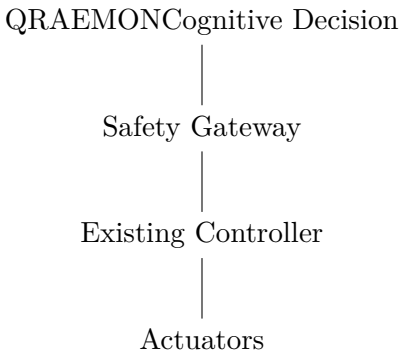


Figure 5: Safety authority remains between cognition and physical execution.

If QRAEMON stops, the existing robot safety and control systems must remain operational.

15 Failure Handling

Failure is a first-class state.

Failure	QRAEMON Response	Robot Requirement
Sensor timeout	Mark state stale; stop or replan	Controller remains operational
Adapter disconnected	Cancel or hold cognitive actions	Robot enters defined safe state
Low confidence	Reject, defer or request more perception	No uncontrolled execution
Planner failure	Invoke recovery or fail safely	Cancellation supported
Memory unavailable	Continue in degraded mode if safe	Cognitive functions degrade gracefully
Model failure	Fallback to deterministic behavior or safe state	Existing controller remains authoritative
Network loss	Continue local operation where supported	No cloud dependency for core loop
Conflicting memories	Reduce confidence and require stronger evidence	No automatic unsafe action

16 Observability and Decision Tracing

Every cognitive action should produce a trace.

At minimum:

- input context version;
- relevant memory identifiers;
- reasoning/planning version;
- selected action;
- confidence;
- safety decision;
- execution result;

- outcome metrics;
- latency;
- failure information.

This trace is essential because the central research claim is causal:

> Did accumulated experience actually change the decision and improve the outcome?

17 Phase 1 — Simulation

Goal: prove the cognitive architecture before exposing expensive physical hardware to it.
Build:

- simulated robot adapter;
- deterministic sensor/state stream;
- existing-controller simulation;
- task environment;
- QRAEMON cognitive runtime;
- no-memory baseline;
- memory-enabled version;
- deterministic experiment runner.

17.1 Initial Tasks

Use controlled manipulation tasks:

- pick-and-place;
- sorting;
- insertion;
- tool use;
- failure recovery.

The simulation should intentionally introduce repeatable failures.
Examples:

- object displacement;
- grasp failure;
- blocked path;
- incorrect insertion alignment;
- temporary sensor degradation;
- changed object configuration.

18 No-Memory Baseline

This is one of the most important components of the entire project.
Build a robot system with:

- the same robot;
- the same compute;
- the same perception;
- the same task;
- the same reasoning capability;
- no persistent experience advantage.

Then compare it with:

Robot + QRAEMON Memory

under matched experimental conditions.

> **Success Criterion**
>
> The objective is not to prove that QRAEMON is intelligent by demonstration.
> The objective is to measure whether persistent experience produces a statistically and operationally meaningful improvement over the corresponding no-memory baseline.

19 Phase 2 — Memory Proof

The central experiment should measure:

$$\Delta SR = SR_{\text{memory}} - SR_{\text{baseline}}$$

where SR denotes success rate.
Additional metrics:

$$\Delta RR = RR_{\text{memory}} - RR_{\text{baseline}}$$

$$\Delta SE = SE_{\text{memory}} - SE_{\text{baseline}}$$

and transfer improvement.
Where:

- SR = success rate;
- RR = recovery rate;
- SE = sample efficiency.

The experiment must control for:

- compute;
- perception;
- task difficulty;
- training exposure;

- robot hardware;
- execution environment.

20 Memory Ablation Program

At minimum compare:

1. no memory;
2. raw retrieval;
3. selective memory;
4. memory without consolidation;
5. memory without retrieval ranking.

The purpose is to determine which mechanism actually creates the performance improvement.

This converts the memory architecture from a philosophical concept into an experimentally testable engineering system.

21 Phase 3 — Physical MVP

Only after the simulation system is stable should the first physical robot be introduced.

The recommended experimental platform is:

- a 6-DOF collaborative or research manipulator;
- RGB-D perception;
- robot state;
- optional force/torque sensing.

This platform is useful because it provides controlled and observable experimentation while allowing the existing robot controller to retain physical execution authority. :chatgpt-content-referenceindex="2"

SensorsRobot State —— QRAEMONRuntime —— Task / SkillAction —— SafetyGateway —— ExistingContr

Figure 6: Physical QRAEMON Software deployment path.

22 Hardware-in-the-Loop Validation

The first physical tests must constrain:

- maximum velocity;
- maximum acceleration;
- workspace;
- force;
- torque;
- energy;

- allowed action set.

Testing should progress from:

1. simulated robot;
2. hardware-in-the-loop;
3. controlled robot;
4. constrained laboratory tasks;
5. repeated experiments;
6. increasingly realistic operating conditions.

23 Production Software Architecture

Once the cognitive hypothesis is validated, the software must transition from research code to an engineered product.

23.1 Core Components

The production runtime should contain:

- runtime orchestration;
- context engine;
- perception interface;
- memory engine;
- reasoning engine;
- planner;
- decision engine;
- safety gateway;
- robot adapter;
- experience recorder;
- evaluation engine;
- telemetry system;
- configuration system;
- model registry.

23.2 Versioning

Every deployed cognitive component must be versioned.

Examples:

- runtime version;
- memory schema version;
- model version;

- planner version;
- adapter version;
- capability schema version.

24 Testing Strategy

24.1 Unit Tests

Test:

- state normalization;
- coordinate conversion;
- action validation;
- memory retrieval;
- memory consolidation;
- planner outputs;
- safety guards;
- adapter errors.

24.2 Integration Tests

Test:

- state → QRAEMON;
- QRAEMON → adapter;
- adapter → robot stack;
- action completion;
- action failure;
- sensor dropout;
- controller dropout;
- cancellation;
- recovery.

24.3 Regression Tests

Every newly discovered failure should become a reproducible regression test where practical.

This is especially important for memory-driven behavior.

25 Security Architecture

Even though this document concerns the software product, security must be designed from the beginning.

Requirements include:

- authenticated communication;

- explicit authorization;
- signed software packages;
- signed model packages;
- secure configuration;
- protected credentials;
- audit logs;
- controlled updates;
- rollback capability.

The future embedded product will additionally require secure boot, device identity and protected persistent storage. :chatgpt-content-referenceindex="3"

26 Deployment Engineering

The software product should support multiple deployment modes.

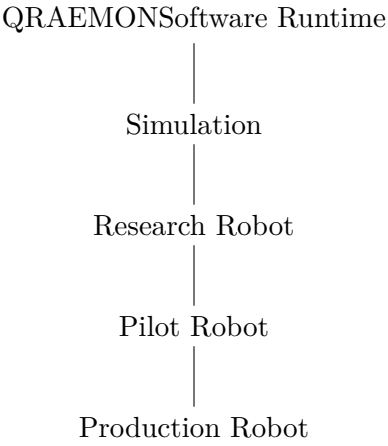


Figure 7: Deployment maturity path.

The same cognitive runtime should progressively move from experimental environments toward controlled production deployment.

27 Developer SDK

A successful software product cannot depend entirely on the internal QRAEMON team.

An SDK should eventually allow external engineering teams to:

- define robot adapters;
- describe robot capabilities;
- register skills;
- define action constraints;
- access experience APIs;
- inspect cognitive traces;

- run simulations;
- execute evaluation suites.

The SDK becomes an important part of the platform moat.

28 Evaluation Platform

A serious robotics intelligence company needs its own evaluation system.

The platform should maintain:

- benchmark tasks;
- experiment configurations;
- robot configurations;
- memory configurations;
- model versions;
- experiment results;
- failure traces;
- statistical analysis.

Every major software change should be evaluated against historical benchmarks.

29 Continual Learning

The long-term software objective is not periodic retraining.

The system should gradually evolve through experience.

A simplified lifecycle is:

Experience —— Memory —— Knowledge —— BetterDecision —— NewExperience

Figure 8: Continual experience-to-improvement cycle.

The first versions may implement this conservatively.

The long-term architecture should allow increasingly sophisticated consolidation, abstraction and adaptation.

30 Cross-Task Transfer

Once single-task memory works, the next research question is transfer.

Example:

- experience from one insertion task;
- experience from another object;
- experience from another orientation;
- experience from another environment.

The system should determine whether previous experience can improve the new task.

This is where memory begins moving from episodic storage toward reusable knowledge.

31 Cross-Robot Experience

The long-term software architecture should allow experience to transfer across compatible robot embodiments.

The adapter should translate embodiment-specific details.

The cognitive layer should retain the higher-level experience.

> **Core Principle**
>
> The long-term moat is not merely the robot adapter.
> It is the combination of cognitive architecture, experience representation, outcome data, memory algorithms, evaluation infrastructure and years of deployment experience.

32 External Pilot Program

After the physical MVP demonstrates measurable memory advantage, begin external technical pilots.

Potential early environments:

- university robotics laboratories;
- industrial research laboratories;
- manufacturing automation;
- controlled manipulation environments;
- warehouse robotics research;
- inspection robotics.

The first pilot should not attempt to solve every robotics problem.
It should test one proposition:

> *Does QRAEMON reduce repeated operational failures by allowing a robot to use previous experience?*

33 Productization

Once the core technology works, transform the research system into a product.

Product components:

1. QRAEMON runtime;
2. robot adapter SDK;
3. memory engine;
4. experience database;
5. evaluation platform;
6. deployment tooling;
7. monitoring;
8. diagnostic tools;

9. simulation package;
10. documentation;
11. update system;
12. enterprise security.

34 Software Product Packaging

The initial commercial software should have three conceptual layers.

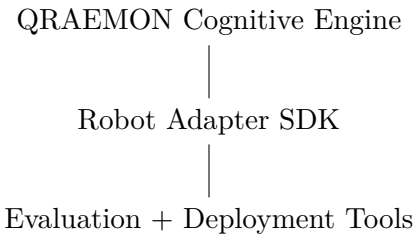


Figure 9: Commercial software stack.

This separation allows the cognitive engine to remain the central proprietary asset.

35 Final Software Roadmap

The complete development sequence is:



Figure 10: QRAEMON Software development gates.

These are engineering gates rather than promises.

35.1 Gate 1 — Architecture

Must have:

- canonical state schema;
- canonical action schema;
- adapter SDK;
- capability model;
- action lifecycle;
- safety interface;
- experience schema.

35.2 Gate 2 — Simulation

Must have:

- end-to-end cognitive loop;

- deterministic tasks;
- no-memory baseline;
- memory-enabled system;
- repeatable experiments.

35.3 Gate 3 — Physical MVP

Must have:

- stable robot adapter;
- safe physical execution;
- reproducible manipulation tasks;
- complete decision traces.

35.4 Gate 4 — Experience Proof

Must demonstrate:

- measurable success-rate improvement;
- improved recovery;
- improved sample efficiency;
- useful transfer;
- statistically credible results.

35.5 Gate 5 — Pilot

Must demonstrate:

- reliable deployment;
- repeatable operation;
- external technical validation;
- useful operational experience.

35.6 Gate 6 — Productization

Must have:

- stable runtime;
- SDK;
- deployment system;
- monitoring;
- security;
- documentation;
- supportability;
- versioning and rollback.

36 Definition of Software Success

QRAEMON Software succeeds technically when all of the following are true:

1. A robot can operate through the QRAEMON cognitive runtime.
2. The QRAEMON core remains robot-agnostic.
3. New robots can be integrated through adapters.
4. Experiences are represented structurally.
5. Relevant experiences can be retrieved.
6. Important experiences can be strengthened.
7. Unnecessary experiences can be compressed or forgotten.
8. Decisions can be traced.
9. Failures can be recovered from.
10. Memory-enabled performance beats the no-memory baseline.
11. Improvements survive repeated trials.
12. Improvements can transfer across tasks.
13. The software remains safe when cognitive components fail.
14. The runtime can eventually operate locally inside the embedded product.

Success Criterion

The strongest definition of success is not that QRAEMON can perform a task.
The strongest definition is that, after experiencing a task repeatedly, QRAEMON becomes measurably better at making decisions about that task without requiring complete retraining of the entire system.

37 Core Software Metrics

The engineering organization should maintain a small set of primary metrics.

Metric	Meaning	Desired Direction
Success Rate	Fraction of tasks successfully completed	Increase
Recovery Rate	Fraction of failures successfully recovered	Increase
Sample Efficiency	Performance improvement per unit experience	Increase
Transfer	Benefit of previous experience on new tasks	Increase
Memory Efficiency	Useful improvement per unit stored experience	Increase
Latency	Time required to make a cognitive decision	Decrease

Reliability	Successful runtime operation over long periods	Increase
Safety Violations	Unsafe actions or constraint violations	Zero

38 Immediate Engineering Backlog

The first implementation sequence should be:

1. Freeze the canonical QRAEMON architecture.
2. Define the canonical robot-state schema.
3. Define the canonical action and skill schema.
4. Define the capability model.
5. Define action lifecycle and cancellation semantics.
6. Define the safety-gateway contract.
7. Implement the adapter SDK.
8. Build a simulated robot adapter.
9. Implement the first memory API.
10. Implement context construction.
11. Implement a minimal planner.
12. Implement experience recording.
13. Implement outcome evaluation.
14. Build the no-memory baseline.
15. Build the memory-enabled experimental system.
16. Build deterministic benchmarks.
17. Establish latency and reliability measurements.
18. Run memory ablations.
19. Only then select the first physical robot configuration.

This ordering follows the architecture-first, simulation-first, evidence-first development path. :chatgpt-content-referenceindex="4"

39 What Not to Build First

QRAEMON should deliberately avoid several attractive distractions.

- A humanoid robot.
- A giant proprietary foundation model.
- Custom AI silicon.
- A custom robot platform.

- Twenty robot integrations.
- A consumer chatbot.
- A generic RAG platform.
- Cloud infrastructure as the core product.
- Autonomous low-level motor control.
- A giant memory database without outcome evaluation.

The first objective is to prove the core cognitive hypothesis.

40 Transition to QRAEMON Embedded

The software architecture should be designed so that the same cognitive core can eventually run on dedicated onboard hardware.

The software product therefore becomes the experimental and technological foundation for the embedded product.

The shared components should include:

- cognitive runtime;
- memory architecture;
- experience representation;
- reasoning APIs;
- canonical robot state;
- action schema;
- safety interface;
- adapter SDK;
- evaluation system.

The major difference is deployment and hardware ownership rather than cognition. :chatgpt-content-referenceindex="5"

41 Long-Term Software Moat

The strongest software moat should emerge from a compounding loop:

Deploy —— Experience —— Remember —— Learn —— Improve —— BetterDeployment

Figure 11: Compounding QRAEMON software loop.

Every deployment should create new experience.

Every useful experience should improve the system.

Every improvement should increase deployment value.

This creates a potential long-term data and architecture advantage that is difficult to reproduce without equivalent operational history.

42 Final Architectural Position

QRAEMON Software should be understood as the first operational embodiment of the QRAEMON artificial-brain architecture.

Its purpose is not to create another robotics model.

Its purpose is to establish the infrastructure through which a machine can:

$$\boxed{\text{Observe} \rightarrow \text{Remember} \rightarrow \text{Reason} \rightarrow \text{Act} \rightarrow \text{Evaluate} \rightarrow \text{Learn}}$$

and repeat that process indefinitely.

The immediate product is software.

The long-term technology is a persistent cognitive architecture.

The long-term platform principle is:

$$\boxed{\text{One Cognitive Architecture} \longrightarrow \text{Many Robot Embodiments}}$$

Robot-specific engineering should remain isolated inside adapters and embodiment interfaces.

The software product therefore serves two purposes simultaneously:

1. It is the first commercializable QRAEMON product.
2. It is the research and engineering foundation for the future QRAEMON Embedded artificial-brain computer.

Core Principle

North Star

QRAEMON Software should make a robot capable of accumulating operational experience and using that experience to make better future decisions, while remaining safe, observable, robot-agnostic and continuously deployable.

QRAEMON

Observe → Remember → Reason → Act → Learn → Evolve